

Plug-and-Play Image Restoration with Deep Denoiser Priors : IRCNN vs. DRUNet under HQS splitting algos across Multiple Inverse Problems Introduction of IRCNN+, a mix between IRCNN and DRUNet

Paul-Emile Barranger, Joris Olivier

CentraleSupélec – Deep Learning Project, January 2026

Abstract

Image restoration has been studied for decades through model based formulations and optimization techniques, by solving inverse problems of the form $y = Hx + n$ with different regularizers. In the context of this Deep Learning course project, we really wanted to understand what modern deep learning techniques can add to this field of computer vision and how modern learning based methods can strengthen principled optimization schemes, rather than fully replacing them with end-to-end black-box models.

Motivated by this question, we explored the Plug-and-Play (PnP) literature, where a learned denoiser replaces an explicit regularizer within iterative solvers, enabling the same algorithmic "framework" to handle multiple inverse problems by only changing the forward operator. In this project, we implement a PnP framework and compare three Deep denoisers : (i) **IRCNN** (bank of CNN denoisers trained at discrete noise levels with σ between 2 and 50), (ii) **DRUNet** (a residual U-Net conditioned on a noise level map) and a (iii) **IRCNN+** (a noise-map conditioned variant of IRCNN) which is a novelty from our part. We implement from scratch and retrain the 3 deep denoisers. Then, we evaluate the methods on four restoration tasks : denoising, deblurring, super-resolution (SISR), and inpainting using PSNR and SSIM. In the literature, IRCNN and DRUNet have not been tested on inpainting tasks yet. We also look at the convergence of our PnP algorithm for the 4 different tasks : PSNR and SSIM over iterations, relative updates, and consistency between HQS variables.

1. Introduction and Context

Image restoration problems are commonly written as $y = Hx + n$, where x is the unknown clean image, H a degradation operator (identity, blur, downsampling, masking, ...), n an additive noise term and y is the observed image. From a Bayesian perspective, the solution can be obtained by solving a Maximum A Posteriori (MAP) estimation problem, which yields this composite objective :

$$\hat{x} = \arg \min_x \underbrace{\frac{1}{2} \|y - Hx\|_2^2}_{\text{data-fidelity}} + \lambda \underbrace{\phi(x)}_{\text{prior}}. \quad (1)$$

There are two main approaches to resolve this kind of problem. On the one hand, model-based methods solve an explicit optimization problem and can handle many inverse problems (denoising, deblurring, super-resolution, inpainting) by simply changing the forward operator H and choosing explicit regularizers (Total Variation etc...). On the other hand, learning-based methods train a neural network (or other ML methods) end-to-end to map a degraded observation y directly to a clean estimate $\hat{x} = f(y; \theta)$. They are typically fast at test time, but require task specific training (a different model for each task we want to

achieve).

Plug-and-Play (PnP) bridge these two worlds : a single denoiser is trained once, and then reused as an implicit prior inside an iterative optimization algorithm. As a result, by only changing the degradation operator H , the same PnP-HQS framework can address a wide range of image restoration tasks (denoising, deblurring, SISR and inpainting).

What did we do concretely in this project.

- Implemented and trained from scratch the denoiser priors **IRCNN**, **IRCNN+** and **DRUNet**
- Implemented a PnP framework for **4 inverse problems** (denoise, deblur, SISR, inpaint) and evaluated with different metrics on natural images.
- Convergence diagnostics : $\text{PSNR}(x_k)$, $\text{PSNR}(z_k)$, $\text{SSIM}(x_k)$, $\|x_{k+1} - x_k\| / \|x_0\| \dots$

2. Framework : Plug-and-Play and HQS

Half Quadratic Splitting (HQS). There exist various variable splitting algorithms for plug-and-play IR, but HQS owes its popularity to the simplicity and fast convergence so we adopt it in this project as it is the one they use in

the papers studied (Zhang et al. (2017) and Zhang et al. (2021)). Starting from (1), we introduce an auxiliary variable z to decouple data term and prior :

$$\mathcal{L}_\mu(x, z) = \frac{1}{2} \|y - Hx\|_2^2 + \lambda\phi(z) + \frac{\mu}{2} \|z - x\|_2^2. \quad (2)$$

As $\mu \rightarrow +\infty$, minimizers of (2) approach those of (1).

Alternating minimization yields :

$$x^{k+1} = \arg \min_x \frac{1}{2} \|y - Hx\|_2^2 + \frac{\mu_k}{2} \|x - z^k\|_2^2 \quad (3)$$

$$z^{k+1} = \arg \min_z \frac{\mu_k}{2} \|z - x^{k+1}\|_2^2 + \lambda\phi(z). \quad (4)$$

In PnP, the z -update is replaced by a deep denoiser :

$$z^{k+1} = D_{\sigma_k}(x^{k+1}), \quad \sigma_k \propto \sqrt{\lambda/\mu_k}. \quad (5)$$

You can find the general algo of PnP using HQS and learned denoisers as priors :

Algorithm 1: General algo PnP-HQS

Input: y, H , denoiser family D_σ , schedule $(\mu_k)_{k=0}^{K-1}$, initialization z^0

for $k = 0$ **to** $K - 1$ **do**

$$\begin{cases} x^{k+1} \leftarrow (H^\top H + \mu_k I)^{-1} (H^\top y + \mu_k z^k); \\ \sigma_k \leftarrow \sqrt{\lambda/\mu_k}; \\ z^{k+1} \leftarrow D_{\sigma_k}(x^{k+1}); \end{cases}$$

Output: $\hat{x} \approx z^K$

The iterations involve μ and λ and the number of outer steps K . As we saw, large μ encourages $x^k \approx z^k$ but typically requires more iterations. Thus a good strategy is to use an increasing sequence $\mu_1 < \dots < \mu_K$, or equivalently a decreasing denoising schedule $(\sigma_k)_{k=1}^K$ through $\sigma_k = \sqrt{\lambda/\mu_k}$. Rather than tuning μ_k directly, we prescribe σ_k by uniformly sampling in log-space from a large σ_1 to a small σ_K (often $\sigma_1 \approx 49$ and σ_K set by the observation noise), and then set $\mu_k = \lambda/\sigma_k^2$.

3. Deep denoiser priors : IRCNN, IRCNN+ and DRUNet

Before presenting the learned denoisers, it is important to remind the reader that we are dealing with non blind restoration setting : the degradation operator H (e.g blur kernel, downsampler, mask) and the observation noise level σ are known, as in standard PnP evaluations. At each iteration, the denoiser is called with an explicit noise level σ_k (set by the chosen schedule), either by selecting the closest IRCNN expert or by providing a sigma-map corresponding to the true noise level observed to DRUNet/IRCNN+. We are aware that estimating H and σ from real data is an important practical problem, but it is not the focus of this study.

3.1. IRCNN Architecture

IRCNN : residual CNN denoisers with discrete noise levels. The backbone of IRCNN denoisers is a 7 layers fully convolutional built with 3×3 dilated convolutions. They are trained for a set of discrete noise levels. A trade-off has to be found between a sufficient number of denoisers to cover well the noise range and the computation associated to adding a specialized noise level model. In our project, we follow the choice of the authors : a bank of 25 specific denoisers for discrete noise levels ranging from $\sigma = 2$ to $\sigma = 50$. Here is the architecture of the IRCNN composed of 7 layers alternating Convolutions, ReLU and Batch Normalizations.

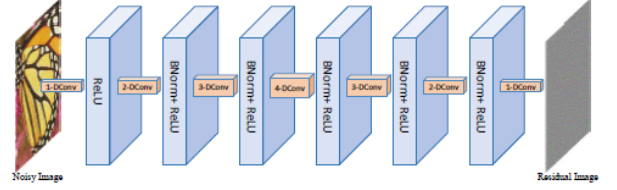


FIGURE 1 – Architecture of the IRCNN denoiser.

Note that each middle layer contains 64 features maps. Another noticeable point is the use of dilated convolutions that allow the network to achieve a large receptive field of 33×33 without increasing network depth or filter size that would add parameters and computational burden. To better understand their advantage, let's consider the example of a convolution with a dilation factor of $s = 2$. It transforms our standard 3×3 filter, composed of weights $(w_i)_{1 \leq i \leq 9}$ to a 5×5 grid by inserting one zero between each weight :

$$W_2 = \begin{bmatrix} w_1 & 0 & w_2 & 0 & w_3 \\ 0 & 0 & 0 & 0 & 0 \\ w_4 & 0 & w_5 & 0 & w_6 \\ 0 & 0 & 0 & 0 & 0 \\ w_7 & 0 & w_8 & 0 & w_9 \end{bmatrix}$$

Thus, the convolutional layer observes a larger spatial area.

3.2. IRCNN Training protocol and project setting

The model employs a residual learning strategy : it learns the noise and retrieve the restaured image by subtracting the degrading one in the forward operation. To do so, it minimizes the ℓ_2 loss between the predicted noise and the ground-truth, corrupted with a gaussian noise $n \sim \mathcal{N}(0, \sigma^2)$, via Adam optimization :

$$l(\Theta) = \frac{1}{2N} \sum_{i=1}^N \|f(y_i; \Theta) - (y_i - x_i)\|^2$$

where $\{(y_i, x_i)\}_{i=1}^N$ represents N noisy-clean patch pairs. We use 400 natural images from BSDS300 database, that we augment via flips and rotations.

Also, to significantly reduce the computational cost of training 25 separate networks from scratch, we implement

a sequential transfer learning strategy. A single model is initialized and trained for $\sigma = 2$, and its optimized weights are subsequently used as the starting point for the next noise level ($\sigma = 4$), continuing up to $\sigma = 50$.

For each expert, the training uses 35×35 patches with a batch size of 8, running for up to 5 epochs of 500 steps. We use the Adam optimizer with a fixed learning rate of 10^{-3} . To prevent accelerate the overall loop, we also introduce a dynamic early stopping mechanism : training for a specific σ stops if the relative improvement in the average loss drops below 1.5% (tolerance = 0.015).

3.3. DRUNet Architecture

The backbone of DRUNet is a residual U-Net whose input is conditioned on the Gaussian noise level via a sigma-map. As given a noisy observation $y = x + n$ with $n \sim \mathcal{N}(0, \sigma^2)$, the network input is the 4-channel tensor $\text{inp} = \text{concat}(y, M_\sigma)$, where M_σ is a spatially uniform map filled with the normalized value $\sigma/255$. Indeed, as we said earlier we do non-blind denoising here so we know in advance the noise level of the input. This enables a single model to handle a wide range of noise levels unlike IRCNN and be more computationnally efficient. We need only to train one model that will be able to handle a wide variety of noise.

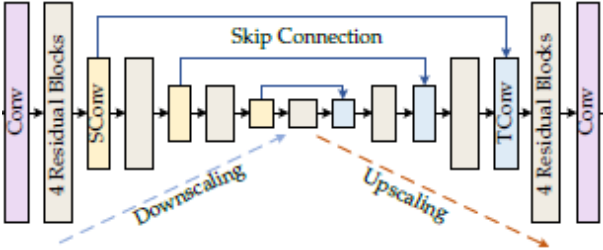


FIGURE 2 – Architecture of the DRUNet denoiser.

The network follows a 4 scale U-Net with channel widths (64, 128, 256, 512) and three downsampling and upsampling stages. At each scale, the representation is refined by residual blocks in order to increase modeling capacity. The first and last layers are bias-free 3×3 convolutions and no activation is applied after these layers. Downsampling is performed with 2×2 strided convolutions (SConv) and upsampling with 2×2 transposed convolutions (TConv), again without activations right after these scaling layers. Each scale contains 4 residual blocks so that there is a single ReLU per block and all convolutional layers are implemented without biases (`bias=False`). Skip connections are implemented by concatenating encoder features with decoder features at matching scales, followed by a 3×3 fusion convolution. To handle arbitrary image sizes, inputs are padded with reflect padding to the next multiple of 8 (due to the three downsampling operations) and unpadded after inference.

3.4. DRUNet Training protocol and project setting

Unlike IRCNN which is trained in a residual way to predict the noise, DRUNet is trained to directly predict the clean image. We train our DRUNet with random 128×128 patches from clean images with the data augmentations as IRCNN (flips, and rotations). For each patch, a noise level is drawn uniformly $\sigma \sim \mathcal{U}(0, 50)$ (pixel scale) and is added $y = x + \varepsilon$. The network is trained with an ℓ_1 loss $\|f_\theta(y, M_\sigma) - x\|_1$ using Adam (batch size 16). The ℓ_1 loss is more robust to outliers. While the original paper (Zhang et al. (2017)) uses a step decay schedule for learning rates, we employed a practical plateau based learning-rate reduction (switching from 10^{-4} to a smaller rate when the loss saturates) and checkpoint the model after each epoch.

3.5. IRCNN+ (our novelty)

IRCNN+ (sigma-map conditioned). We had the idea to “mix” both paradigms to test how a single conditioned IRCNN behaves across a continuous range of noise levels, and whether it can match the flexibility of DRUNet without relying on expert selection. We name it IRCNN+ (be careful because in the DRUNet paper (Zhang et al. (2021)), they call the IRCNN+ the discret bank of denoiser IRCNN but using the same iterative optimization scheme as DRUNet). To sum up, IRCNN+ keeps the same 7-layer dilated CNN backbone as IRCNN, but replaces the bank of experts by a single network conditioned on the noise level just like DRUNet does.

IRCNN+ training. IRCNN+ is trained in the same non-blind Gaussian denoising setting as DRUNet (noise level is provided), but keeps IRCNN’s *residual learning* philosophy (see the code for more details).

4. Experiments

Code : Our GitHub for the project

4.1. General protocol, dataset, and metrics

As said earlier, all experiments are performed in a non-blind setting which means that the degradation model (operator H , blur kernel, mask...) and the observed noise level are assumed to be known.

We use natural images from the BSDS300 dataset, with training and evaluation performed on the corresponding **train** and **test** splits. For each restoration task, we have 2 “pipelines” to evaluate the methods. First, we can run the full pipeline on a single image (and for a specific task) and save qualitative and visual outputs (the clean image, the degraded one and the restored) together with convergence curves. Second, we can run a benchmark over a pool of $N = 10$ test images and report mean and standard deviation of our metrics (for a specific task as well).

Moreover, IRCNN is implemented as a bank of 25 noise-specific experts with $\sigma \in \{2, 4, \dots, 50\}$, so during inference

we select the closest expert for the current σ_k that we will use in the iterative optimization scheme. In contrast, DRUNet and IRCNN+ uses a single network conditioned by a sigma-map channel. Except this difference, the pipeline for the different tasks is exactly the same. We will call "denoiser" our DRUNet, or IRCNN, or IRCNN + for generality in all the following part but it can refer to DRUNet or IRCNN as it is the same pipeline.

4.2. Gaussian denoising

For denoising, the degradation model is simply additive white Gaussian noise. For the DRUNet, given y and σ , the denoiser is applied once by concatenating the noisy RGB image with a constant noise-level map : $\text{inp} = \text{concat}(y, M_\sigma)$ and $\hat{x} = D_{\text{DRUNet}}(\text{inp})$. In our implementation, noisy images are generated without clipping before denoising to preserve AWGN statistics. For IRCNN, denoising is performed by selecting the expert network corresponding to the noise level σ and applying it once with $\hat{x} = D_{\text{IRCNN}}^{(\sigma)}(y)$. For the benchmark, we sample $N = 10$ test images for both methods, add noise with a fixed σ , denoise each image, and store per-image PSNR and SSIM.

4.3. Deblurring (with Levin09 kernels)

For deblurring, we use spatially invariant blur with additive Gaussian noise $y = k * x + n$ and $n \sim \mathcal{N}(0, \sigma^2)$. We use the same blur kernels from the Levin09 collection as the authors (that we loaded from `Levin09.npy`) and implement the forward blur using circular convolution in the Fourier domain as described in Drunet paper (Zhang et al. (2021)). The inverse problem is solved : at each iteration k , we alternate a closed-form quadratic x -update in the Fourier domain and a denoising z -update with the denoiser. The denoiser noise level σ_k follows a log spaced schedule from a large value (e.g., 49) down to σ_{real} , and the penalty parameter is set accordingly. Finally, we compare the blurred and noisy observation y to the restored image $\hat{x}$ using PSNR and SSIM, and we run the same pipeline over a random pool of $N = 10$ images.

4.4. Inpainting

For inpainting, we generate a binary mask $M \in \{0, 1\}^{H \times W}$ (random rectangular missing regions) and form the masked observation $y = M \odot x + n$ where noise is optionally added on observed pixels only. We use an analytic Shepard-style interpolation as initialization z_0 : missing pixels are filled by a weighted average of neighboring known pixels within a fixed window. Starting from this initialization, we run PnP-HQS with the denoiser as the prior. The x -update is pixelwise closed-form due to the diagonal structure of M , while the z -update is a denoising call. After each denoiser step, we enforce data consistency by re-inserting known pixels :

$$z \leftarrow M \odot y + (1 - M) \odot z.$$

4.5. Single Image Super-Resolution

For SISR with scale factor s , we use the classical blur-downsample-noise model :

$$y = S(k * x) + n,$$

where S is decimation by factor s and k is a known blur kernel (loaded from `kernels_12.mat`). We implement the forward model with circular boundary conditions and generate the low-resolution observation y , optionally adding AWGN. The reconstruction follows : the x -update is computed by the standard closed-form frequency-domain solution for the blur+downsample quadratic subproblem, and the z -update is still the denoising step. The initialization z_0 is bicubic upsampling.

5. Results and Interpretations

5.1. Global quantitative comparison (PSNR and SSIM on 10 images)

Tables .1 and .2 report the average PSNR and SSIM over a pool of $N = 10$ test images for the four tasks (denoising, deblurring, SISR, and inpainting). Overall, DRUNet consistently achieves the best performance across tasks, while IRCNN remains competitive on some problems but is a bit limited due to its shallow layers, and our IRCNN+ shows mixed behavior depending on the problem.

Denoising. For $\sigma = 25$, DRUNet has the highest PSNR and SSIM (Table .1 and Table .2). This superiority is expected. Indeed, beyond the choice of being trained once for a whole range of noise levels through sigma-map conditioning, DRUNet relies on a more modern and expressive residual U-Net architecture, which provides a stronger image prior than a CNN backbone. IRCNN remains relatively close, showing that a bank of noise specialized experts can still be competitive when the test noise level is well covered by the discrete training grid. IRCNN+ slightly lags behind in PSNR. Indeed, with same architecture, IRCNN benefits from noise-level specialization, while IRCNN+ must "share parameters" across all noise levels. Although parameter sharing can slightly hurt pixel-wise accuracy (as seen with PSNR), it can act as a form of regularization that favors perceptually relevant features, which are better captured by SSIM and Table .2 shows it with a slightly better SSIM for IRCNN+.

Deblurring. DRUNet still exhibits a strong gain over IRCNN and IRCNN+ in both metrics. This is consistent with the PnP-HQS mechanism where the iterates contain structured, non-Gaussian artifacts produced by the data step, then with a denoiser with stronger expressive power and multi-scale context (the DRUNet), it removes these artifacts more effectively. We also see that IRCNN+ performs slightly better (both with SSIM and PSNR) than IRCNN on average, but with larger variance on PSNR, hinting maybe at less stable behavior.

Single Image Super-Resolution (SISR). DRUNet again leads on average. Notably, IRCNN+ is competitive, maybe indicating that conditioning can help preserve perceptual structure.

Inpainting. As we said, we tried our denoiser in a new IR task that has not been tested in the literature. DRUNet is clearly the best, while IRCNN provides only a modest improvement over the Shepard initialization in PSNR but not in SSIM. In contrast, IRCNN+ performs significantly worse in both metrics, showing that in this task the sigma-conditioned shallow CNN can fail to act as a reliable prior and may even amplify artifacts or undo the initialization structure.

5.2. Qualitative results : visual comparison by task

We now complement the numerical evaluation with visual comparisons on two representative images (CASTEL and PLANE), reported in Appendix Annexe A. The goal is to visually assess texture recovery, artifact suppression, and failure modes that may not be fully captured by PSNR or SSIM.

5.2.1. Gaussian denoising and generalization beyond the training range

First of all, for a moderate noise level ($\sigma = 20$) A.4, all three methods produce visually satisfying restorations and the differences are little, which is expected since the denoising problem remains relatively easy and close to the training distribution. To better separate behaviors, we increase the noise to the maximum where they have been trained $\sigma = 50$ A.5, and the advantage of DRUNet becomes clear with smoother result, fewer residual grain artifacts. IRCNN and especially IRCNN+ struggle to fully remove noise and may leave structured residuals.

We further test a challenging out of range setting with $\sigma = 100$ A.6, even though all denoisers were trained up to $\sigma \leq 50$. In this regime, DRUNet remains effective, indicating strong generalization. In contrast, IRCNN and IRCNN+ fail : their outputs remain heavily corrupted and do not match the proper denoising behavior.

5.2.2. Deblurring : artifact suppression and stability

DRUNet provides the best visual restoration, typically yielding fewer structured artifacts. IRCNN also produces reasonable restorations, but one can observe faint residual artifacts. Importantly, IRCNN+ tends to amplify these artifacts into visible oscillatory patterns (wavelets in the sky or the lake) A.14, which degrades the visual quality even when the reconstruction remains globally plausible. This failure mode suggests that the sigma-map conditioning in a shallow CNN can behave more aggressively in the presence of structured deconvolution noise, whereas DRUNet better absorbs these patterns due to its stronger multiscale prior.

5.2.3. SISR : all methods are strong but IRCNN can preserve fine textures

The three methods provide high-quality results and the differences are way less pronounced than in deblurring A.18. On the CASTEL image, IRCNN appears slightly more precise on certain fine details that DRUNet does not catch (real “waves” on the lake surface) even though the average PSNR and SSIM said the inverse. It looks like it preserves very small textures.

5.2.4. Inpainting : strong dependence on the denoiser prior

For inpainting, the Shepard initialization already produces a visually pretty clean image, meaning the PnP algorithm mainly needs to refine details and enforce natural image statistics. Here, DRUNet clearly improves the result further, sharpening details and producing a more coherent completion. A.20 IRCNN brings only a very small improvement over the initialization, indicating limited ability to refine entire missing regions in this setting. IRCNN+ behaves very poorly : it tends to revert toward a more “masked” appearance. This confirms the quantitative observation that IRCNN+ is not robust.

6. Convergence diagnostics

This section compares the convergence behaviors of the HQS optimization scheme using our three learned priors. The consistency distance $\text{RMSE}(x_k - z_k)$ between x_k (data fidelity term) and z_k (denoising with learned prior) will validate the HQS splitting of (2). The PSNR and SSIM evolutions will tell about the success of our restoration. Finally the relative change ($\frac{\|x_k - x_{k-1}\|}{\|x_0\|}$) and cumulated sum ($\sum_{k=1}^N \frac{\|x_k - x_{k-1}\|}{\|x_0\|}$) of x_k and z_k will validate the stability and convergence of our algorithm.

6.1. Deblurring

For the deblurring task, all three models successfully A.23, ?? and ?? minimize the consistency distance $\text{RMSE}(x_k - z_k)$. Their PSNR and SSIM have the same shape : a monotonical raise before reaching a stable plateau, reaching good restoration performances. However, their internal dynamics differ significantly. DRUNet is stable and smooth with the most robust convergence as shown by relative updates that decrease steadily toward zero (A.25). On the contrary, IRCNN has many relative updates oscillations, reflecting its discrete nature with transitions between specific noise-level experts. To finish, IRCNN+ resolves the discrete instabilities of the standard IRCNN due to its continuous structure, but it suffers from a late stage conflict between the regularizer (at low noise levels) and the data fidelity step A.29.

6.2. SISR

Unlike the monotonic convergence observed in the deblurring task, SISR curves reveal an early peak before

a slow degradation for the data fidelity term x_k across all models (like A.30). It means that the restored image reaches its maximum structural fidelity long before the final iteration and that prolonged optimization forces the solution to drift away from the exact ground truth. The three models have the same specific behaviors as deblurring due to their architecture.

6.3. Inpainting

The inpainting task introduces difficulties for our algorithms with enforcement of known pixels in the data fidelity step. DRUNet manages well this test with great PSNR and SSIM performances, while keeping a smooth convergence. The chaotic behavior of IRCNN is emphasized and even lead to non convergence as the relative changes show (A.28). The algorithm remains resilient and obtains reasonable metrics. If the IRCNN+ still eliminates the discrete transition shocks of the IRCNN, it entirely fails the inpainting task as illustrated by decreasing PSNR and SSIM. The rebound in relative change confirms the algorithmic rejection of a solution.

7. Conclusion

This project explored Plug-and-Play image restoration by implementing and training the DRUNet, IRCNN, and IRCNN+ models entirely from scratch. We evaluated these deep priors within our HQS framework across four inverse problems : denoising, deblurring, SISR, and inpainting. Quantitatively, the evaluation metrics globally favored the DRUNet model, which provided highly stable convergence and superior overall performance. In contrast, the standard IRCNN exhibited discrete structural shocks, and our attempt to smooth these transitions via IRCNN+ revealed a trade-off : its continuous regularization caused over-smoothing under strict data-fidelity constraints like in inpainting.

Ultimately, our convergence diagnostics permit to understand better each model's behavior. If convergence is almost always reached (except for IRCNN and IRCNN+ inpainting task), with more smoothness for DRUNet, the recurring phenomenon of early peaking in PSNR curves lead us to adopt in the future an early stopping strategy. Despite these algorithmic challenges, the final restored images exhibited highly satisfactory visual sharpness and structural recovery across all tasks. This study aligns with current state-of-the-art research : integrating deep learned priors within the PnP-HQS framework provides a robust and highly powerful approach to solve a wide array of ill-posed inverse problems.

Références

Zhang, K., Li, Y., Zuo, W., 2021. Plug-and-play image restoration with deep denoiser prior, in : IEEE Transactions on Pattern Analysis and Machine Intelligence.

Zhang, K., Zuo, W., Gu, S., Zhang, L., 2017. Learning deep cnn denoiser prior for image restoration, in : Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

Results summary

Method	Denoise ($\sigma = 25$)	Deblur (Levin kernel, $\sigma = 5$)	SISR (Levin kernel, $\times 2$)	Inpaint (missing ratio, σ)
Degraded	20.53 $\pm$ 0.26	22.70 $\pm$ 2.29	25.25 $\pm$ 2.59 (Bicubic init)	29.03 $\pm$ 2.90 (Shepard init)
IRCNN	29.64 $\pm$ 1.36	27.39 $\pm$ 1.00	27.80 $\pm$ 2.72	29.80 $\pm$ 1.42
IRCNN+	29.21 $\pm$ 1.45	28.06 $\pm$ 1.98	29.32 $\pm$ 2.80	26.59 $\pm$ 2.18
DRUNet	30.61 $\pm$ 2.08	29.68 $\pm$ 2.40	29.53 $\pm$ 2.91	31.25 $\pm$ 2.91

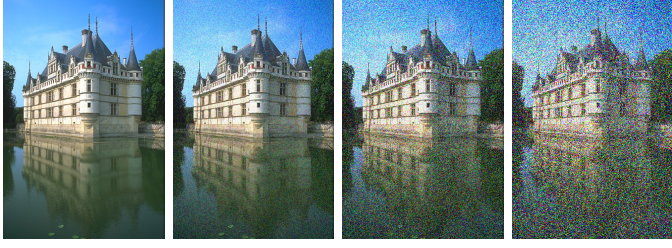
TABLE .1 – Average PSNR on a pool of 10 test images

Method	Denoise ($\sigma = 25$)	Deblur (Levin kernel, $\sigma = 5$)	SISR (Levin kernel, $\times 2$)	Inpaint (missing ratio, σ)
Degraded	0.40 $\pm$ 0.11	0.47 $\pm$ 0.10	0.67 $\pm$ 0.12 (Bicubic init)	0.89 $\pm$ 0.01 (Shepard init)
IRCNN	0.82 $\pm$ 0.04	0.77 $\pm$ 0.04	0.74 $\pm$ 0.12	0.88 $\pm$ 0.03
IRCNN+	0.83 $\pm$ 0.03	0.79 $\pm$ 0.04	0.84 $\pm$ 0.07	0.84 $\pm$ 0.02
DRUNet	0.86 $\pm$ 0.05	0.84 $\pm$ 0.06	0.85 $\pm$ 0.08	0.92 $\pm$ 0.01

TABLE .2 – Average SSIM on a pool of 10 test images

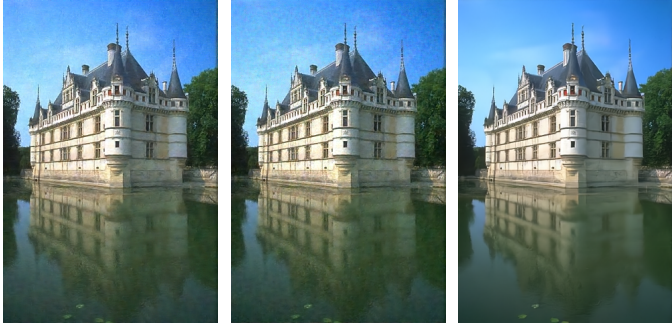
Annexe A. Additional qualitative results

Annexe A.1. Denoising



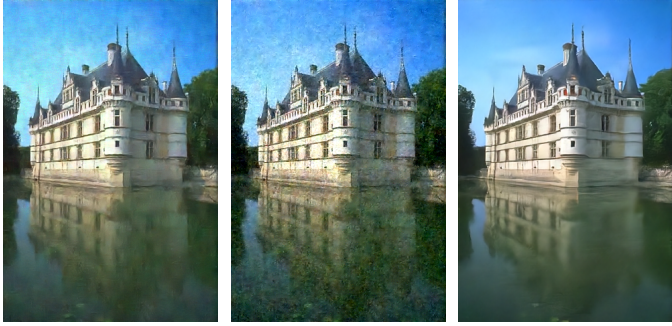
(a) Clean image (b) Noisy image with $\sigma=20$ PSNR = 22.37dB SSIM = 0.38 (c) Noisy image with $\sigma=50$ PSNR = 14.99dB SSIM = 0.16 (d) Noisy image with $\sigma=100$ PSNR = 10.37dB SSIM = 0.07

FIGURE A.3 – Clean and noisy images for CASTEL image



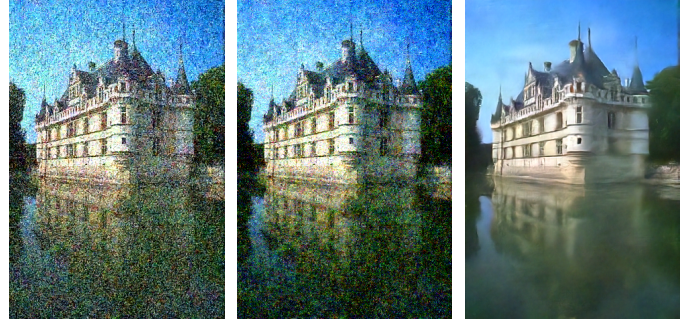
(a) Denoised image with IRCNN PSNR = 32.00dB SSIM = 0.86 (b) Denoised image with IRCNN+ PSNR = 30.90dB SSIM = 0.84 (c) Denoised image with DRUNET PSNR = 32.9dB SSIM = 0.91

FIGURE A.4 – Denoising results for $\sigma=20$ CASTEL image



(a) Denoised image with IRCNN PSNR = 27.20dB SSIM = 0.74 (b) Denoised image with IRCNN+ PSNR = 24.54dB SSIM = 0.62 (c) Denoised image with DRUNET PSNR = 28.66dB SSIM = 0.84

FIGURE A.5 – Denoising results for $\sigma=50$ CASTEL image



(a) Denoised image with IRCNN PSNR = 16.41dB SSIM = 0.19 (b) Denoised image with IRCNN+ PSNR = 16.23dB SSIM = 0.19 (c) Denoised image with DRUNET PSNR = 25.55dB SSIM = 0.76

FIGURE A.6 – Denoising results for $\sigma=100$ CASTEL image



(a) Clean image (b) Noisy image with $\sigma=20$ PSNR = 22.21dB SSIM = 0.33 (c) Noisy image with $\sigma=50$ PSNR = 14.73dB SSIM = 0.11 (d) Noisy image with $\sigma=100$ PSNR = 10.22dB SSIM = 0.04

FIGURE A.7 – Clean and noisy images for PLANE image



(a) Denoised image with IRCNN PSNR = 32.72dB SSIM = 0.83 (b) Denoised image with IRCNN+ PSNR = 31.20dB SSIM = 0.82 (c) Denoised image with DRUNET PSNR = 33.66dB SSIM = 0.85

FIGURE A.8 – Denoising results for $\sigma=20$ PLANE image



(a) Denoised image with IRCNN PSNR = 28.94dB SSIM = 0.73 (b) Denoised image with IRCNN+ PSNR = 24.37dB SSIM = 0.53 (c) Denoised image with DRUNET PSNR = 29.99dB SSIM = 0.77

FIGURE A.9 – Denoising results for $\sigma=50$ PLANE image



(a) Denoised image with IRCNN
PSNR = 16.05dB
SSIM = 0.14

(b) Denoised image with IRCNN+
PSNR = 15.87dB
SSIM = 0.13

(c) Denoised image with DRUNET
PSNR = 26.85dB
SSIM = 0.68

FIGURE A.10 – Denoising results for $\sigma=100$ PLANE image

Annexe A.2. Deblurring



(a) clean image

(b) blurred with Levin09 kernel and $\sigma=5$

FIGURE A.11 – Clean and blurred images PLANE

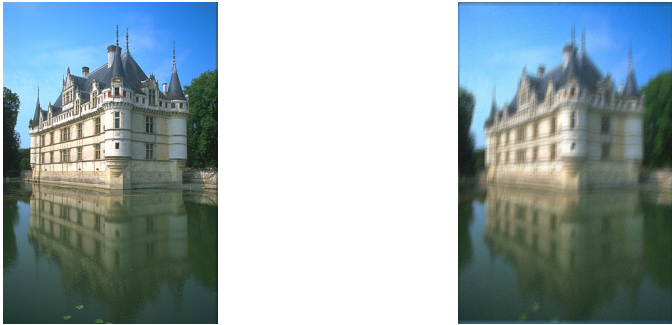


(a) Deblurred image with IRCNN

(b) Deblurred image with IRCNN+

(c) Deblurred image with DRUNET

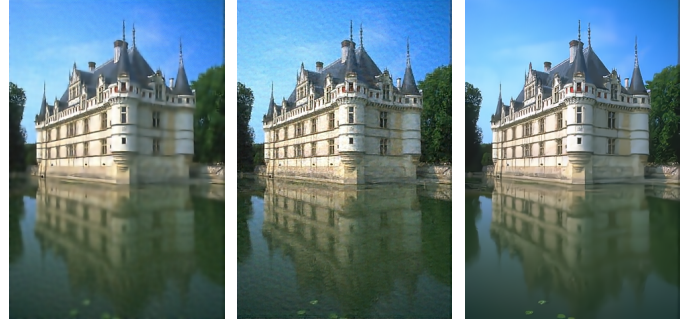
FIGURE A.12 – Deblurring results for PLANE image



(a) Clean image

(b) blurred with Levin09 kernel and $\sigma=5$

FIGURE A.13 – Clean and blurred images CASTEL



(a) Deblurred image with IRCNN

(b) Deblurred image with IRCNN+

(c) Deblurred image with DRUNET

FIGURE A.14 – Deblurring results for CASTEL image

Annexe A.3. Single Image Super-Resolution (SISR)



(a) Clean image

(b) LR image

(c) Bicubic

FIGURE A.15 – Clean and LR images PLANE

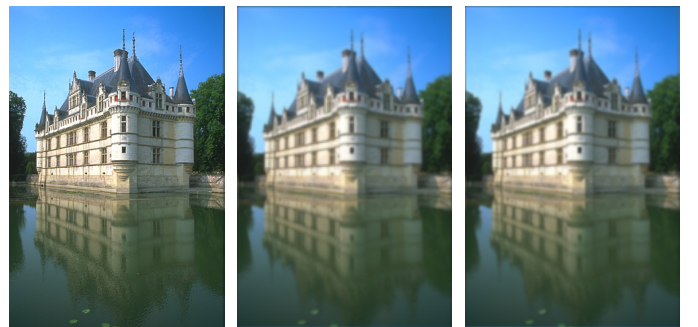


(a) Restored image with IRCNN

(b) Restored image with IRCNN+

(c) Restored image with DRUNET

FIGURE A.16 – SISR results for PLANE image

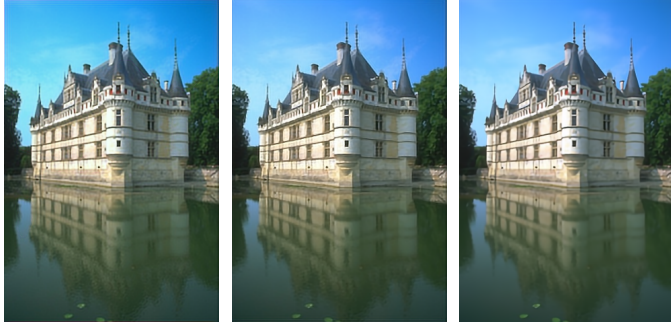


(a) Clean image

(b) LR image

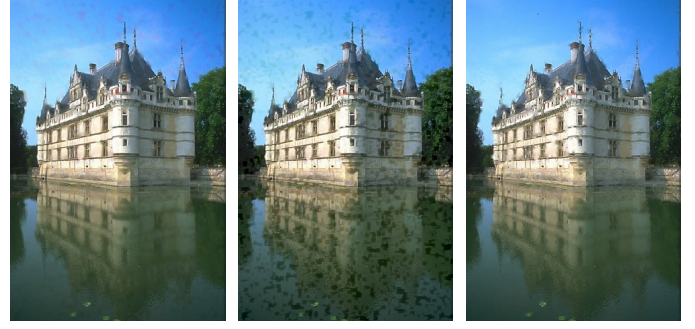
(c) Bicubic

FIGURE A.17 – Clean and LR images CASTEL



(a) Restored image with IRCNN (b) Restored image with IRCNN+ (c) Restored image with DRUNET

FIGURE A.18 – SISR results for CASTEL image.



(a) Restored image with IRCNN (b) Restored image with IRCNN+ (c) Restored image with DRUNET

FIGURE A.22 – Inpainting results for CASTEL image.

Annexe A.4. Inpainting



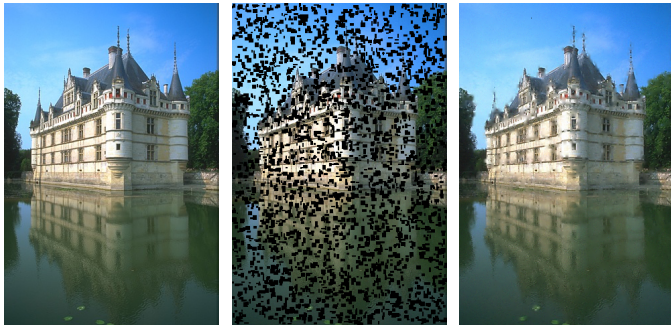
(a) Clean image (b) Masked image (c) Shepard initialization

FIGURE A.19 – Clean and masked images PLANE



(a) Restored image with IRCNN (b) Restored image with IRCNN+ (c) Restored image with DRUNET

FIGURE A.20 – Inpainting results for PLANE image.



(a) Clean image (b) Masked image (c) Shepard initialization

FIGURE A.21 – Clean and masked images CASTEL

Annexe A.5. DRUNET convergence curves

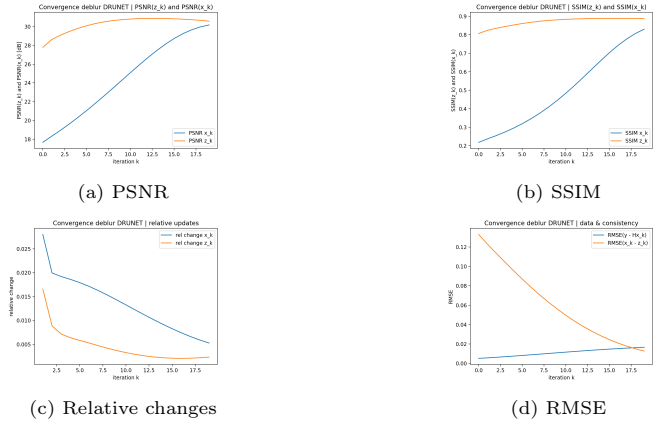


FIGURE A.23 – Deblurring convergence for DRUNET

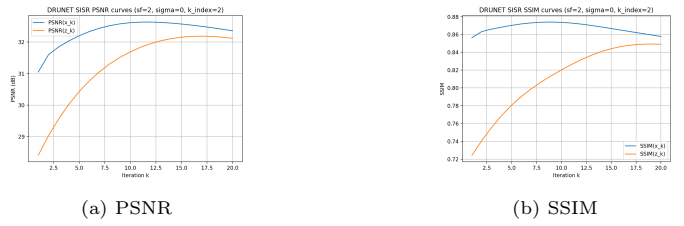
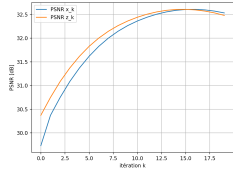
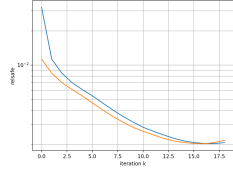


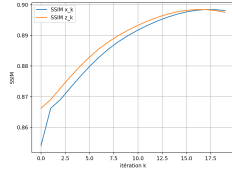
FIGURE A.24 – SISR convergence DRUNET



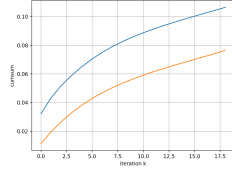
(a) PSNR



(c) Relative change



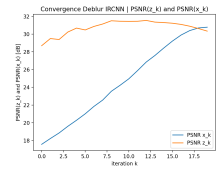
(b) SSIM



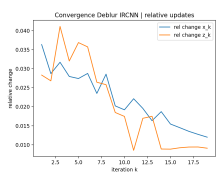
(d) Cumulative sum

FIGURE A.25 – Inpainting convergence DRUNET

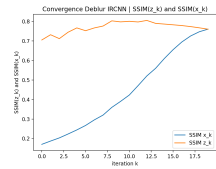
Annexe A.6. IRCNN convergence curves



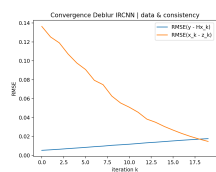
(a) PSNR



(c) relative change

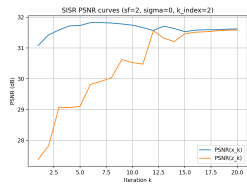


(b) SSIM

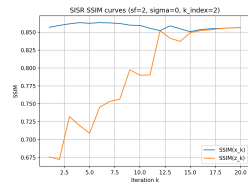


(d) RMSE

FIGURE A.26 – Deblurring convergence for IRCNN

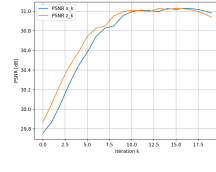


(a) PSNR

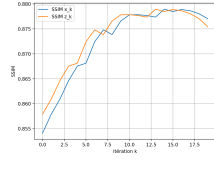


(b) SSIM

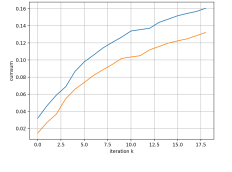
FIGURE A.27 – SISR convrgence IRCNN



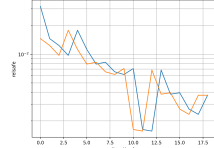
(a) PSNR



(b) SSIM



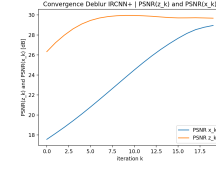
(c) Cumulative sum



(d) Relative change

FIGURE A.28 – Inpainting convergence IRCNN

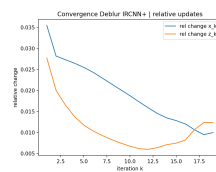
Annexe A.7. IRCNN+ convergence curves



(a) PSNR



(b) SSIM

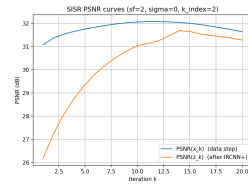


(c) relative change

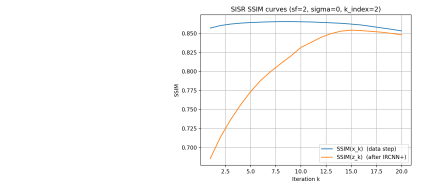


(d) RMSE

FIGURE A.29 – Deblurring convergence for IRCNN+



(a) PSNR



(b) SSIM

FIGURE A.30 – SISR convrgence IRCNN+

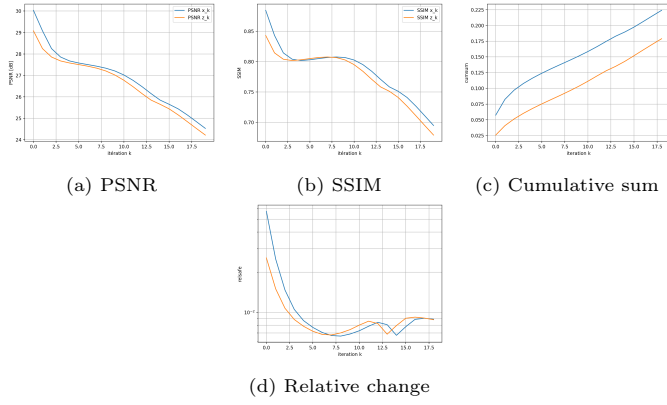


FIGURE A.31 – Inpainting convergence IRCNN+